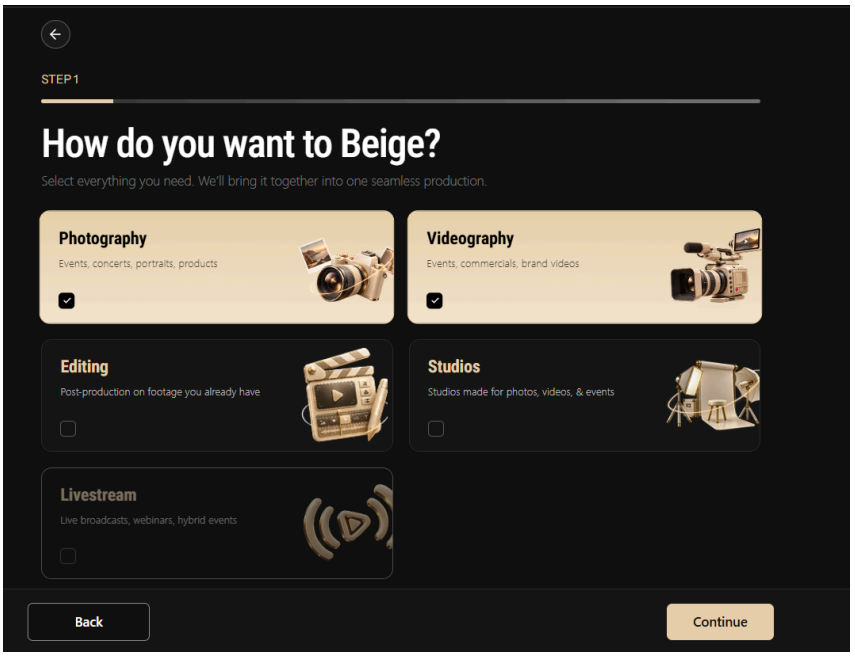
App — New Book a Shoot Flow

All APIs require normal client authorization headers:

Authorization: {{TOKEN}}
device_type: android
user_type_name: client
user_type: 3

The app must keep `booking_id` returned from Screen 1 and use it in every following API.

Screen 1 — Select Content Type



Show these three options:

- Videography
- Photography
- Photography and Videography

When the user taps **Continue**, create or update the draft booking.

POST /bookings

Videography

```
{
  "content_type": "videographer"
}
```

Photography

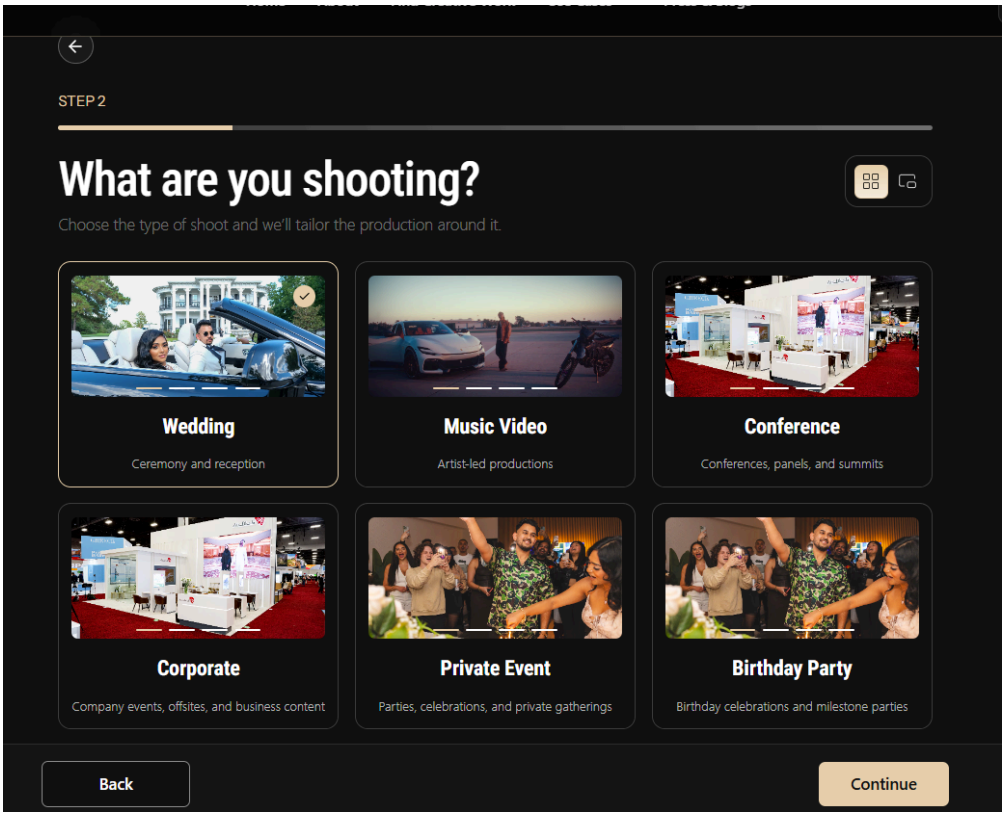
```
{
  "content_type": "photographer"
}
```

Photography and Videography

```
{
  "content_type": "photographer,videographer"
}
```

Save `data.booking_id` from the response.

Screen 2 — Select Shoot Type



Load the shared web shoot-type catalogue when this screen opens.

GET /bookings/shoot-types

This API returns the same shoot types for every content type, for example:

- Wedding
- Music Video
- Conference
- Corporate
- Private Event
- Birthday Party
- Funeral / Memorial
- Real Estate
- Brand / Promotional
- Product
- Other

The API response includes the card name, subtitle/description, image, and display order.

When the user selects a card and taps **Continue**, save the selected shoot type by its name.

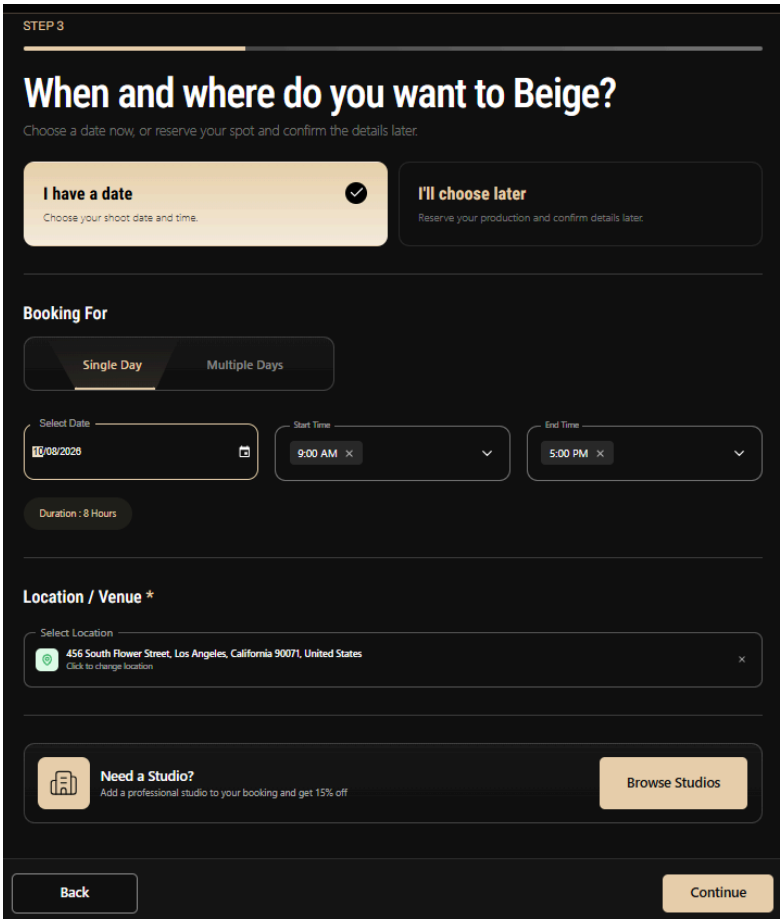
POST /bookings/:bookingId
```
{
  "shoot_type": "Wedding"
}
```

Do not send `shoot_type_id` from mobile. Send the exact `shoot_type` name returned by the catalogue API.

Screen 3 — Date, Time, Timezone, and Location

This screen has two paths.

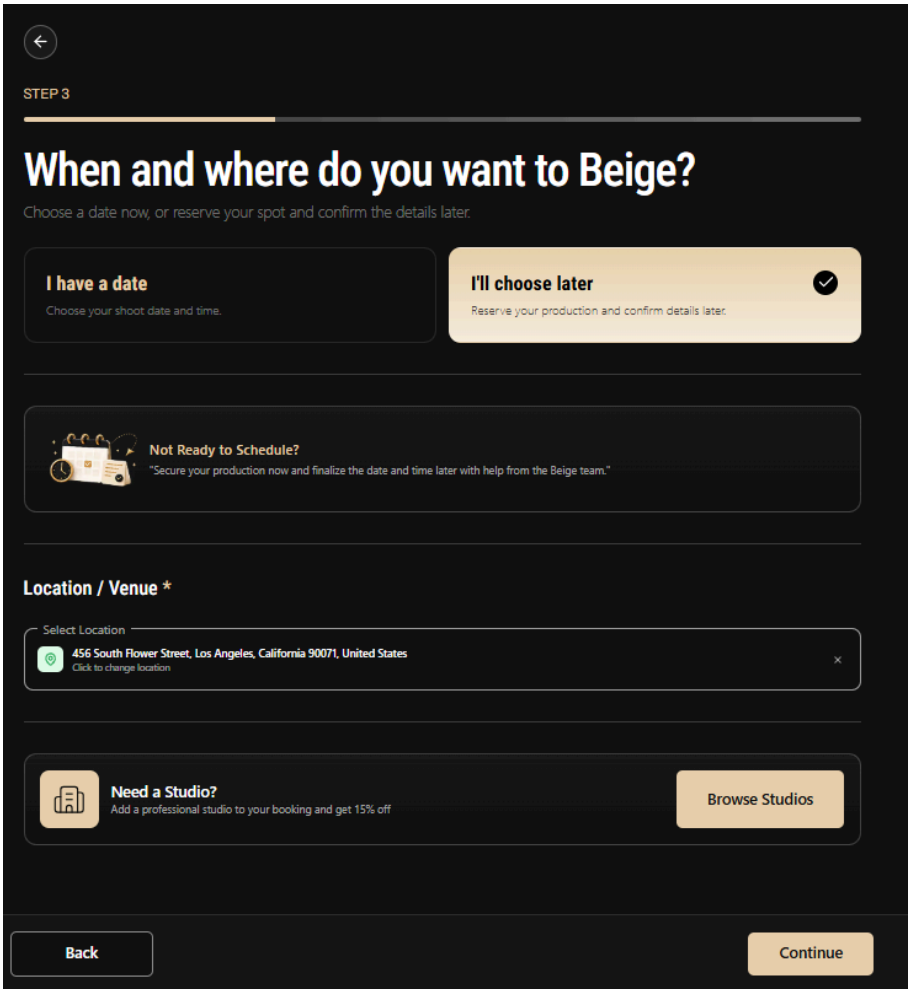
A. User selects “I have a date”



When the user selects date, time, timezone, and location, call:

PUT /bookings/:bookingId/time
{
  "event_date": "2026-10-08",
  "start_time": "09:00",
  "end_time": "17:00",
  "time_zone": "Asia/Calcutta",
  "event_location": "550 West 5th Street, Los Angeles, California 90013, United States",
  "event_latitude": 34.05025748,
  "event_longitude": -118.25407363
}

B. User selects “I’ll choose later”

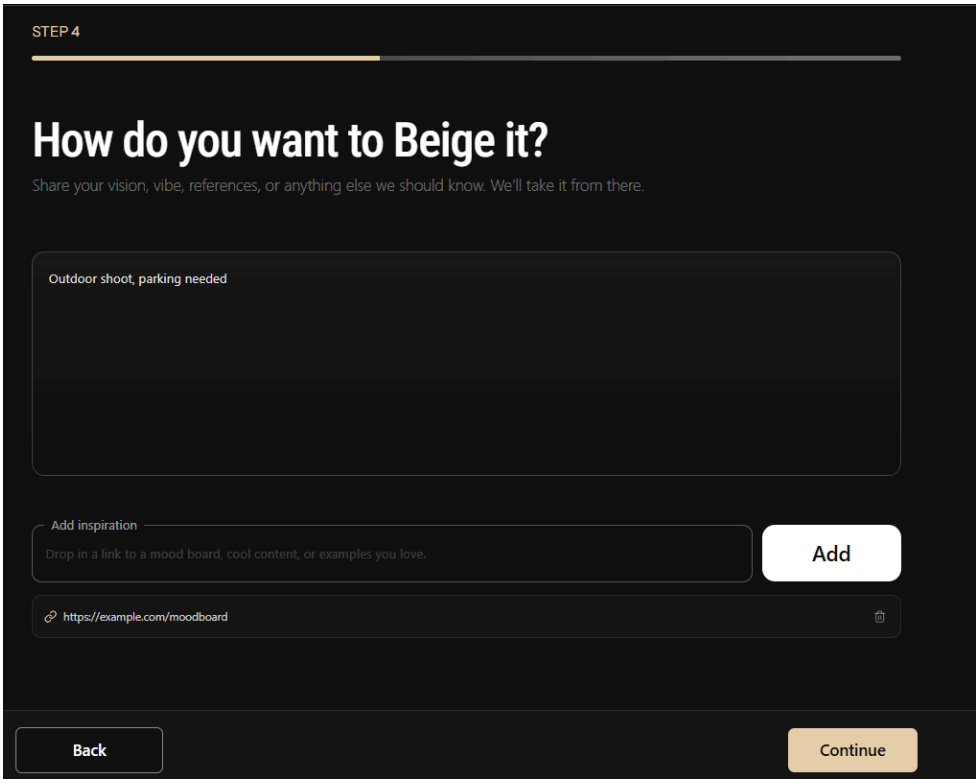


Do not send date or time. Save timezone and location, and explicitly set `confirm_later: true`.

PUT /bookings/:bookingId/time
{
  "confirm_later": true,
  "booking_type": "single_day",
  "booking_days": [],
  "time_zone": "Asia/Calcutta",
  "event_location": "550 West 5th Street, Los Angeles, California 90013, United States",
  "event_latitude": 34.05025748,
  "event_longitude": -118.25407363
}

This booking can still continue to quote and payment without a date/time. The client can provide the final schedule later.

Screen 4 — Additional Details and Reference Links



Show:

- Additional details textarea
- Reference links / inspiration links

When the user taps **Continue**, save these values.

```
PUT /bookings/:bookingId/details
{
  "additional_details": "Outdoor shoot, parking needed",
  "reference_links": [
    "https://example.com/moodboard",
    "https://drive.google.com/example"
  ]
}
```

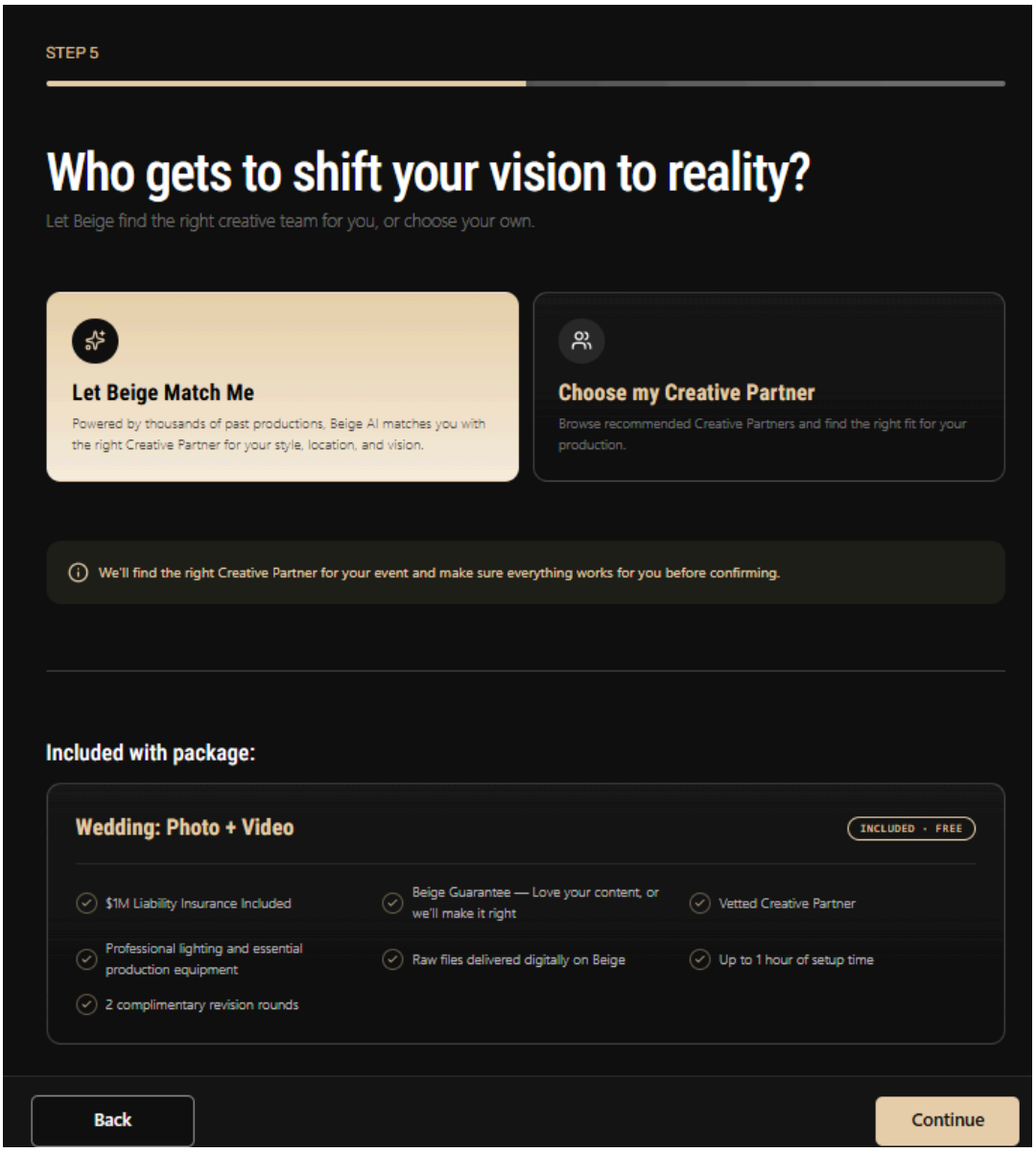
If no links are added, send an empty array:

```
{
  "additional_details": "Outdoor shoot",
  "reference_links": []
}
```

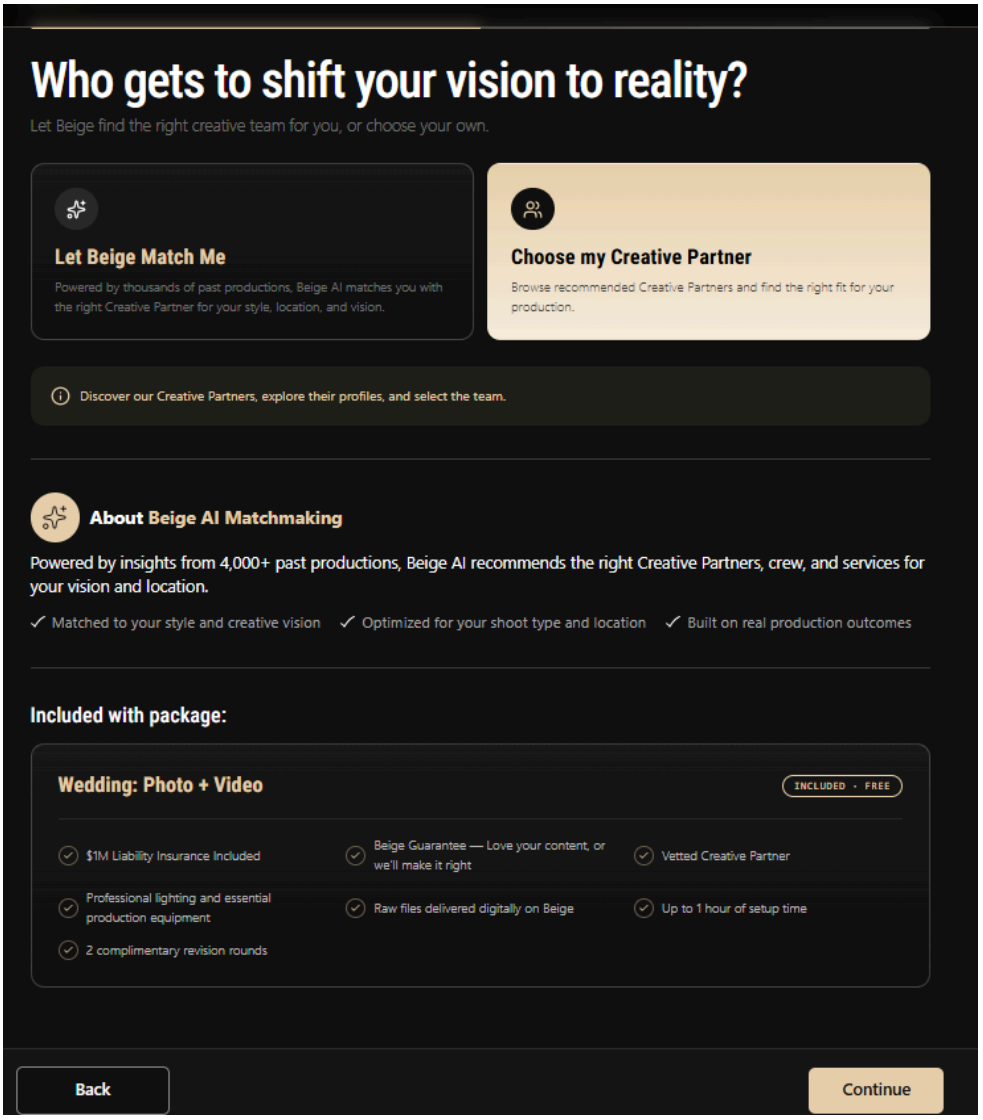
Screen 5 — Matching Choice

Show two options:

1. **Let Beige Match Me**



2. **Choose My Creative Partner**

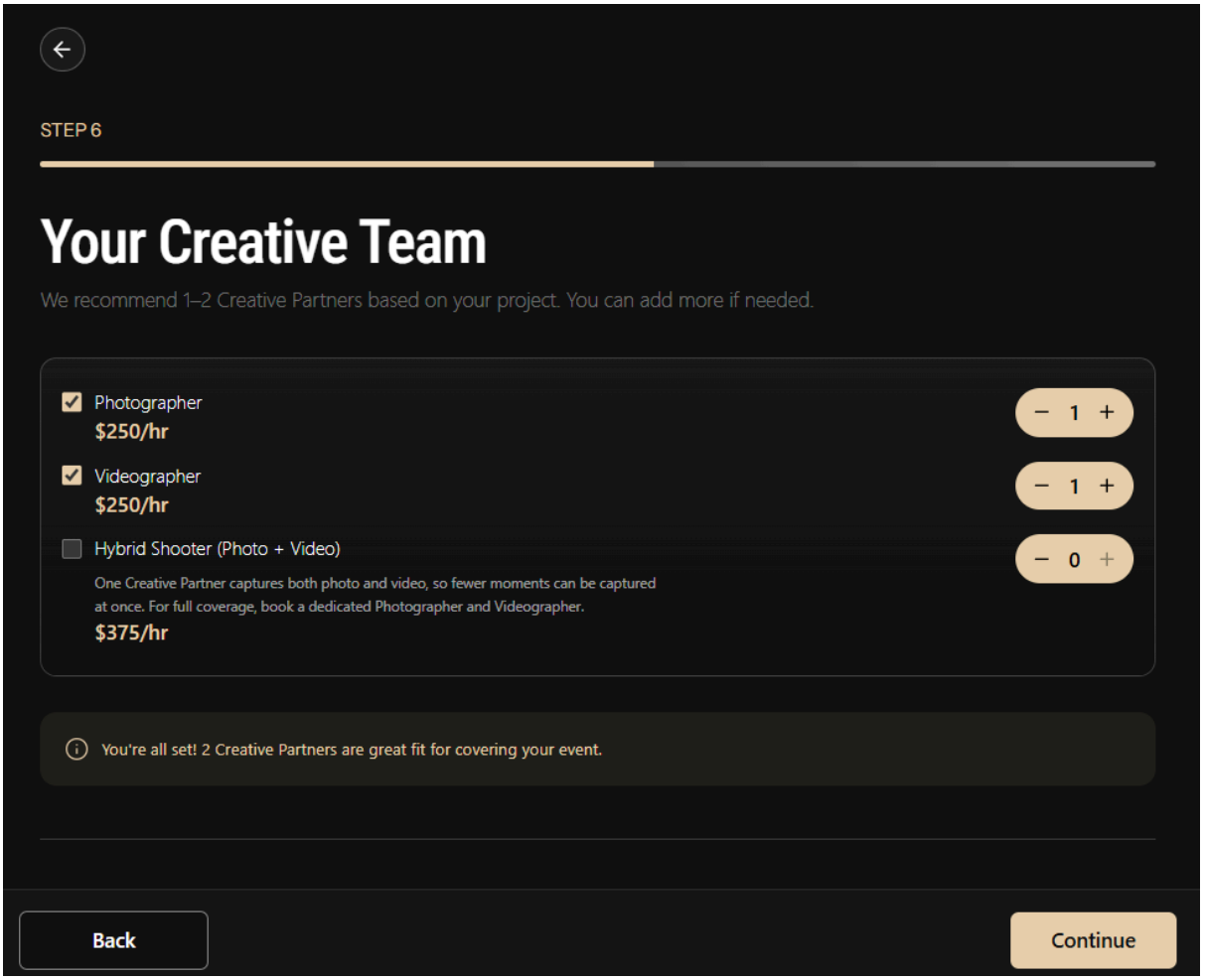


There is **no API call on this screen**.

Keep the selected option in frontend state and use it later on the final submit API.

Both options continue to Screen 6.

Screen 6 — Crew Count



Show role counts:

- Videographer
- Photographer
- Cinematographer
- Photo + Video Creator

When the user taps **Continue**, save the total required creators.

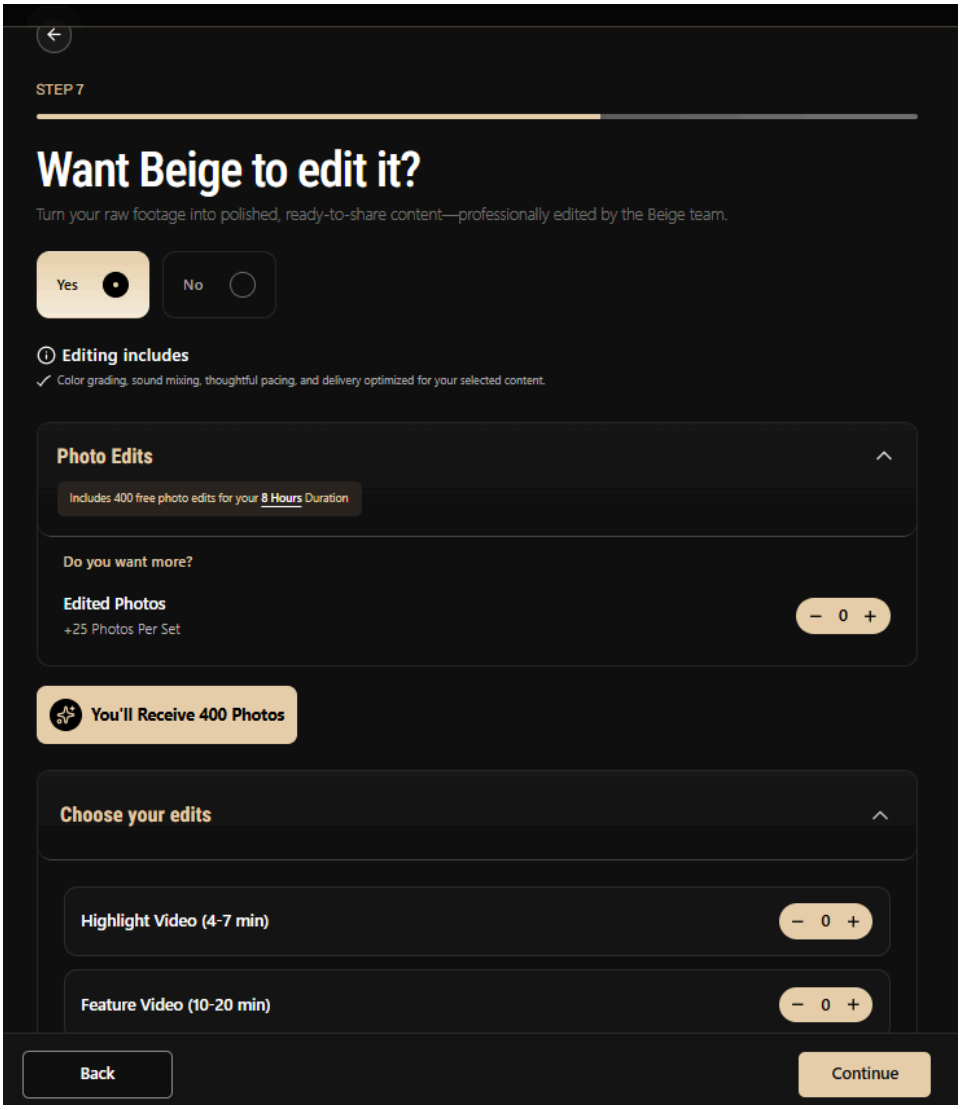
POST /bookings/:bookingId

```
{
  "role_counts": {
    "videographer": 1,
    "photographer": 1,
    "cinematographer": 0,
    "photoVideoCreator": 0
  }
}
```

Important:

- These are the total creators required for the shoot.
- Send `0` for roles not selected.
- Save this before opening the edits screen or creative-partner screen.

Screen 7 — Select Edits



First load edit types based on the selected content type.

Videography

GET /bookings/edit-types?content_type=videographer

Use `video_edit_types` from the response.

Photography

GET /bookings/edit-types?content_type=photographer

Use `photo_edit_types` from the response.

Photography and Videography

GET /bookings/edit-types?content_type=photographer,videographer

Show both `video_edit_types` and `photo_edit_types`.

When the user taps **Continue**, save the selected edit keys.

```
PUT /bookings/:bookingId/edits
{
  "edits_needed": true,
  "video_edit_types": [
    "highlight_video_4_7",
    "feature_video_10_20"
  ],
  "photo_edit_types": [
    "edited_photos"
  ]
}
```

If the user does not need edits:

```
{
  "edits_needed": false,
  "video_edit_types": [],
  "photo_edit_types": []
}
```

Use the exact `key` returned by the edit-types API. Do not create or rename keys in the app.

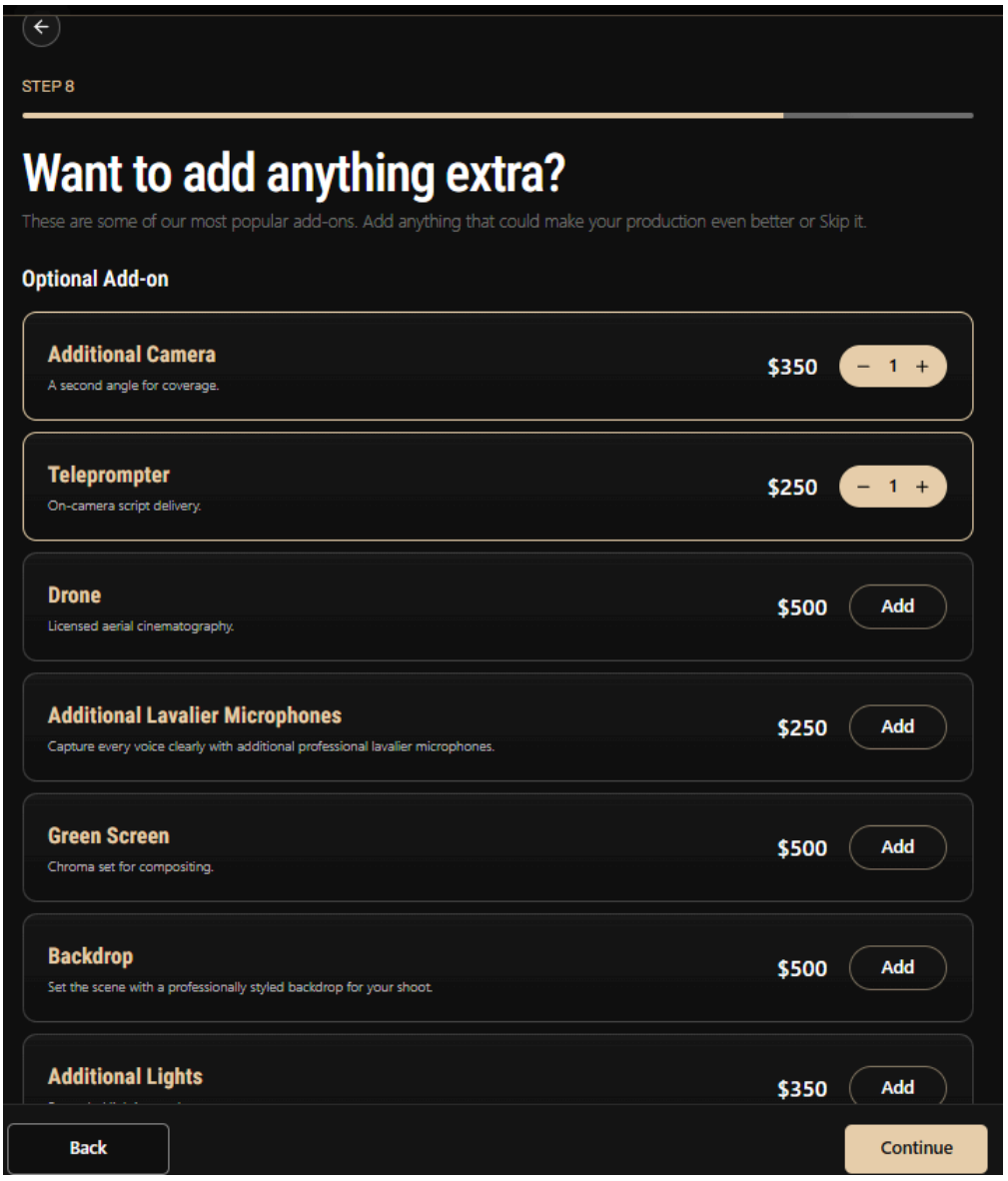
Branch A — Let Beige Match Me

After Screen 7, follow this order:

1. Screen 8 — Add-ons
2. Screen 9 — Review and Contact Information
3. Screen 10 — Confirm and Pay
4. Payment screen

There is no creative-partner selection screen in this branch.

Screen 8 — Add-ons



Load active web-v4 add-ons when the screen opens.

GET /bookings/addons

Keep selected add-ons in frontend state. Do not save them immediately.

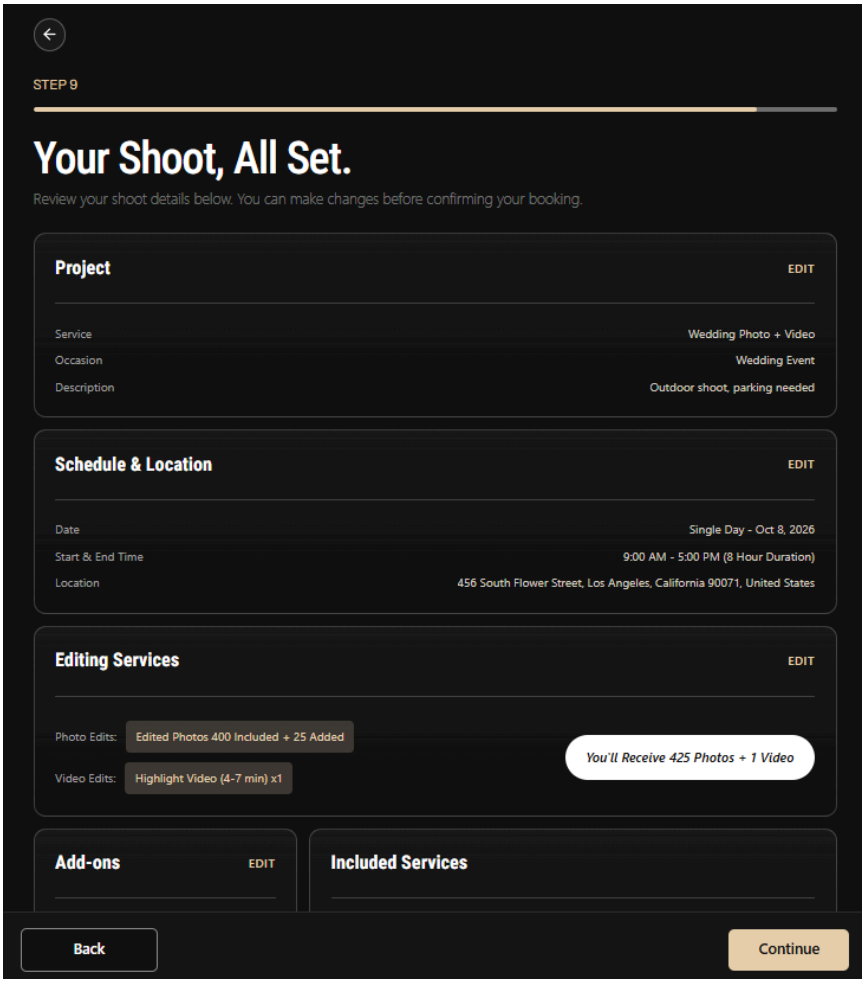
Example frontend state:

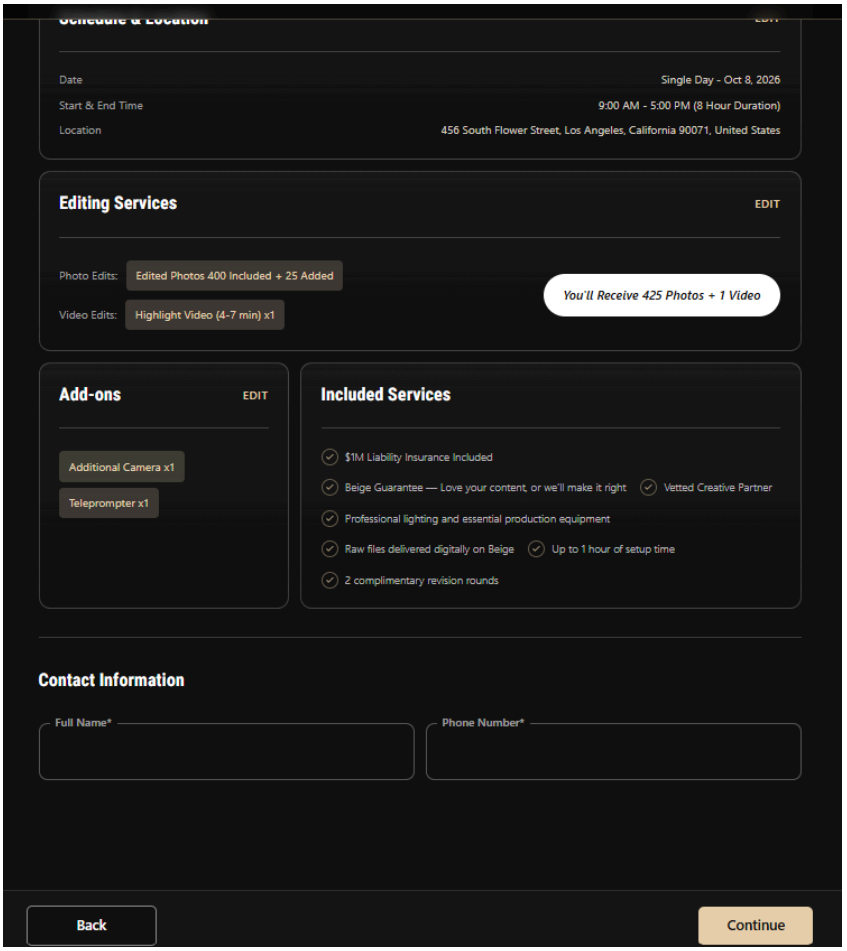
```
[
  {
```

```
    "slug": "v4-additional-camera",
    "quantity": 1
  },
  {
    "slug": "v4-teleprompter",
    "quantity": 1
  }
]
```

The selected add-ons are sent on the Review screen.

Screen 9 — Review and Contact Information





Show:

- Project service
- Occasion
- Description
- Date, time, duration, timezone, and location
- Or “Date to be decided”
- Selected photo edits
- Selected video edits
- “You’ll Receive” text
- Selected add-ons
- Included Services
- Full Name
- Email
- Phone Number

Do not show final pricing, card fee, or payment method on this screen. Those are shown on Screen 10.

Load Screen 9 data

Call this API when Screen 9 opens. Call it again only if the user goes back to Add-ons and changes add-on quantities.

POST /bookings/:bookingId/review

```
{
  "add_on_items": [
    {
      "slug": "v4-additional-camera",
      "quantity": 1
    },
    {
      "slug": "v4-teleprompter",
      "quantity": 1
    }
  ]
}
```

Use response fields directly:

UI field	Response field
Service	`data.project.service`
Occasion	`data.project.occasion`
Description	`data.project.description`
Date / time / duration / location	`data.schedule.*`

Date to be decided	`data.schedule.is_date_to_be_decided`
Photo Edits	`data.editing_services.photo_edits`
Video Edits	`data.editing_services.video_edits`
You'll Receive	`data.editing_services.receive_text`
Add-ons	`data.add_ons`
Included Services	`data.included_services`
Pre-fill Full Name / Email / Phone	`data.contact.*`

Example edit response:


```
{
  "editing_services": {
    "photo_edits": [
      {
        "key": "edited_photos",
        "label": "Edited Photos",
        "quantity": 1,
```

```
      "total_photos": 425
    }
  ],
  "video_edits": [
    {
      "key": "highlight_video_4_7",
      "label": "Highlight Video (4-7 min)",
      "quantity": 1
    }
  ],
  "receive_text": "You'll Receive 425 Photos + 1 Video"
 }
}
```


Full Name, Email, and Phone

Keep the values entered by the user in frontend flow state.

Do not call an API while the user types.

When user taps Continue

Save the quote with the same selected add-ons:

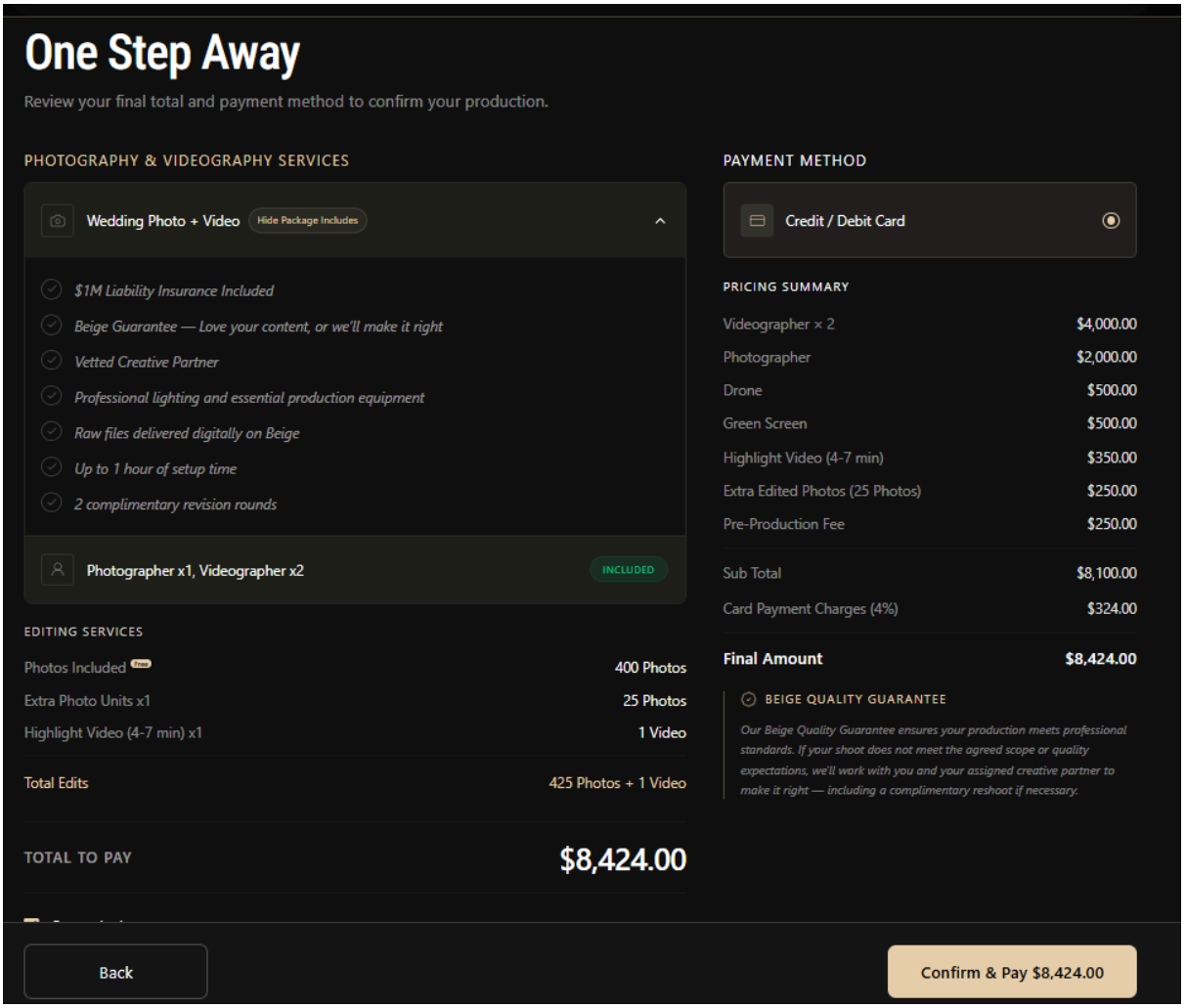
POST /bookings/:bookingId/quote


```
{
 "add_on_items": [
  {
    "slug": "v4-additional-camera",
    "quantity": 1
  },
```

```
    {
      "slug": "v4-teleprompter",
      "quantity": 1
    }
  ]
}
```

After quote save succeeds, open Screen 10 — Confirm and Pay.

Screen 10 — Confirm and Pay



When this screen opens

Call:

GET /bookings/:bookingId/payment-review

Use response fields directly:

UI field	Response field
Service heading	`data.package.service`
Included Services	`data.package.included_services`
Required creator roles	`data.package.crew`
Photo Edits	`data.editing_services.photo_edits`
Video Edits	`data.editing_services.video_edits`
Total Edits	`data.editing_services.receive_text`
Pricing Summary rows	`data.pricing.line_items`
Sub Total	`data.pricing.subtotal`

Card Payment Charges (4%)	`data.pricing.card_processing_fee`
Final Amount / Total To Pay	`data.pricing.final_amount`
Payment method label	`data.payment_method.label`

This screen is read-only. Do not call `/review`, `/pricing/calculate`, or `/quote` again.

When user taps Confirm & Pay

First save final contact information and matching choice:

PUT /bookings/:bookingId/submit

{

"full_name": "Harsh Patel",

"email": "harsh@test.com",

"phone": "7777777777",

"matching_method": "ai_matchmaker",

"selected_crew_ids": []

}

For Let Beige Match Me:

- `selected_crew_ids` must be `[]`.
- Do not call creator hold APIs.
- Beige matches the required creators after booking.

After `/submit` succeeds, open payment:

POST /bookings/:bookingId/paymentsheet

Stripe:

{

"payment_method": "stripe"

}

Commas:

{

"payment_method": "commas"

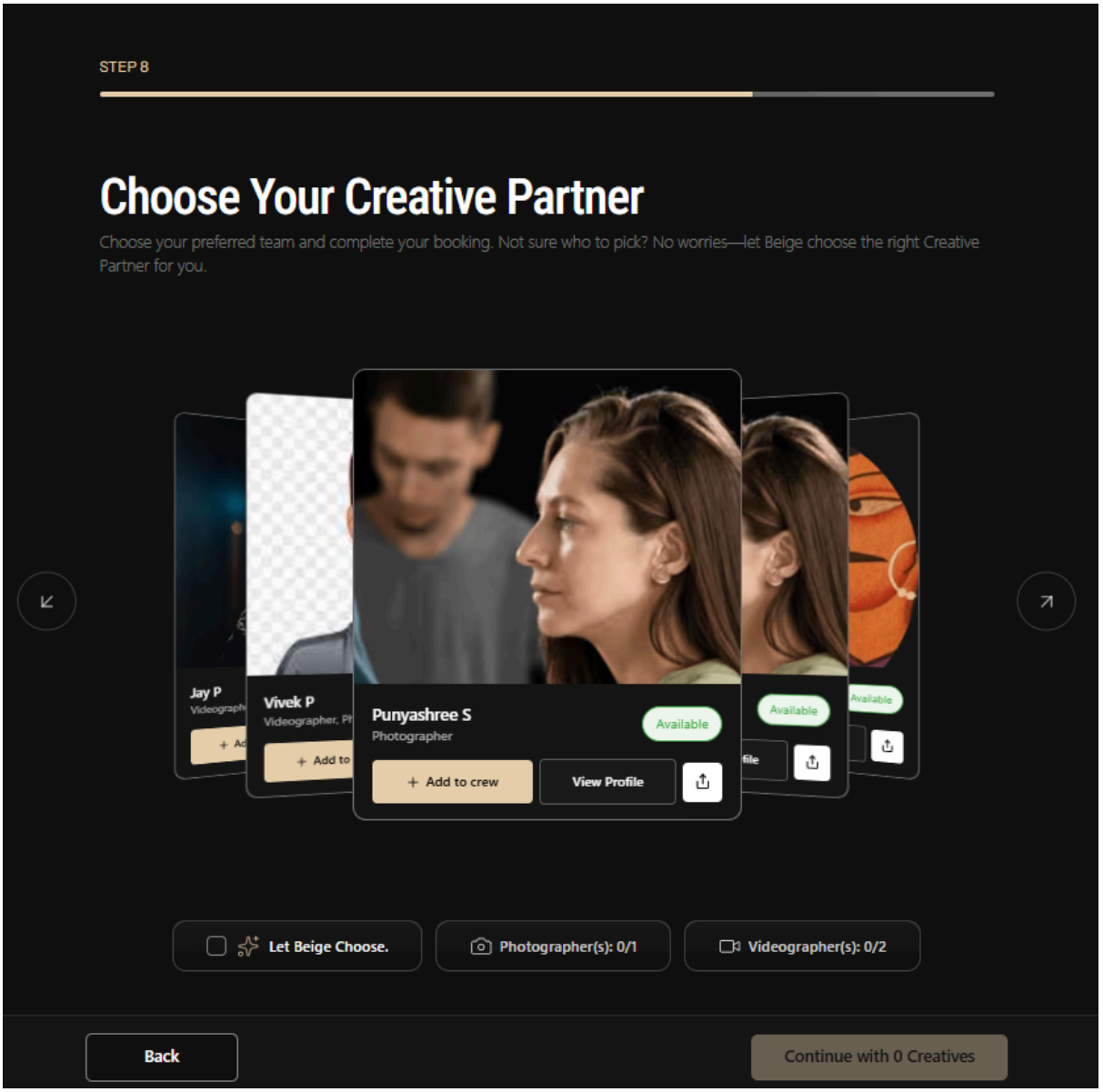
}

Branch B — Choose My Creative Partner

After Screen 7, follow this order:

1. Screen 8 — Choose Your Creative Partner
2. Screen 9 — Add-ons
3. Screen 10 — Review and Contact Information
4. Screen 11 — Confirm and Pay
5. Payment screen

Screen 8 — Choose Your Creative Partner



Load creator cards:

GET /bookings/:bookingId/matches?page=1&limit=12

Use the creator role/capability data returned for each card. Do not decide a creator's role from their name or card title.

The app must keep this frontend-only state:

```
{
  "required_counts": {
    "photographer": 2,
    "videographer": 2
```

```
  },
  "assigned_creators": {
    "photographer": [],
    "videographer": []
  },
  "let_beige_choose": false
}
```

`assigned_creators` must store both the creator ID and the UI role where the creator was placed.

Example:

```
{
  "assigned_creators": {
    "photographer": [
      {
        "crew_member_id": 2129
      }
    ],
    "videographer": [
      {
        "crew_member_id": 2130
      }
    ]
  }
}
```

Show role count buttons

Show one button for every requested role:

Photographer(s): selected / required

Videographer(s): selected / required

Example:

Photographer(s): 1/2

Videographer(s): 2/2

Creator selection logic

Photographer-only creator

If a selected creator supports only Photographer:

Add creator to Photographer count only.

Example:

Photographer(s): 1/2 → 2/2

Videographer-only creator

If a selected creator supports only Videographer:

Add creator to Videographer count only.

Example:

Videographer(s): 0/2 → 1/2

Creator who supports both Photographer and Videographer

If a creator supports both roles, assign them to the role with fewer selected creators.

Example:

Photographer(s): 0/2

Videographer(s): 1/2

Select dual-role creator

→ Assign to Photographer

Another example:

Photographer(s): 2/2

Videographer(s): 1/2

Select dual-role creator

→ Assign to Videographer

If both role counts are equal, assign the dual-role creator to the role with more remaining required slots.

Example:

Photographer(s): 0/2

Videographer(s): 0/1

Select dual-role creator

→ Assign to Photographer, because Photographer has more remaining slots.

If both selected counts and remaining slots are equal, use Photographer as the fixed fallback.

Photographer(s): 0/1

Videographer(s): 0/1

Select dual-role creator

→ Assign to Photographer

If the preferred role is already full, assign the creator to the other supported role if that role still has space.

If all supported role slots are full, disable Add for that creator.

Remove creator logic

When the user removes a creator:

Remove them from the exact role slot where the UI assigned them.

Example:

Selected dual-role creator was assigned to Photographer.

Photographer(s): 2/2

Videographer(s): 1/2

User removes creator

→ Photographer(s): 1/2

→ Videographer remains 1/2

Let Beige Choose checkbox

The **Let Beige Choose** checkbox is frontend/UI state only.

Do not send `let_beige_choose` to backend.

Checkbox is not selected

Enable Continue only when every required role is filled.

Example:

Photographer(s): 2/2

Videographer(s): 2/2

Continue with 4 Creatives

→ Enabled

Example:

Photographer(s): 1/2

Videographer(s): 2/2

Continue

→ Disabled

Checkbox is selected

Enable Continue even if zero creators are selected or some creator slots are still empty.

Example:

Let Beige Choose: selected

Photographer(s): 0/2

Videographer(s): 0/2

Continue with 0 Creatives

→ Enabled

Another example:

Let Beige Choose: selected

Photographer(s): 1/2

Videographer(s): 0/2

Continue with 1 Creative

→ Enabled

Important backend rules

- Do not call hold APIs.
- Do not call availability reservation APIs.

- Do not save selected IDs on this screen.
- Do not send `let_beige_choose` in any API payload.
- Keep selected IDs and UI role assignment in frontend state until final payment submit.

At final Confirm and Pay, send only all selected creator IDs:

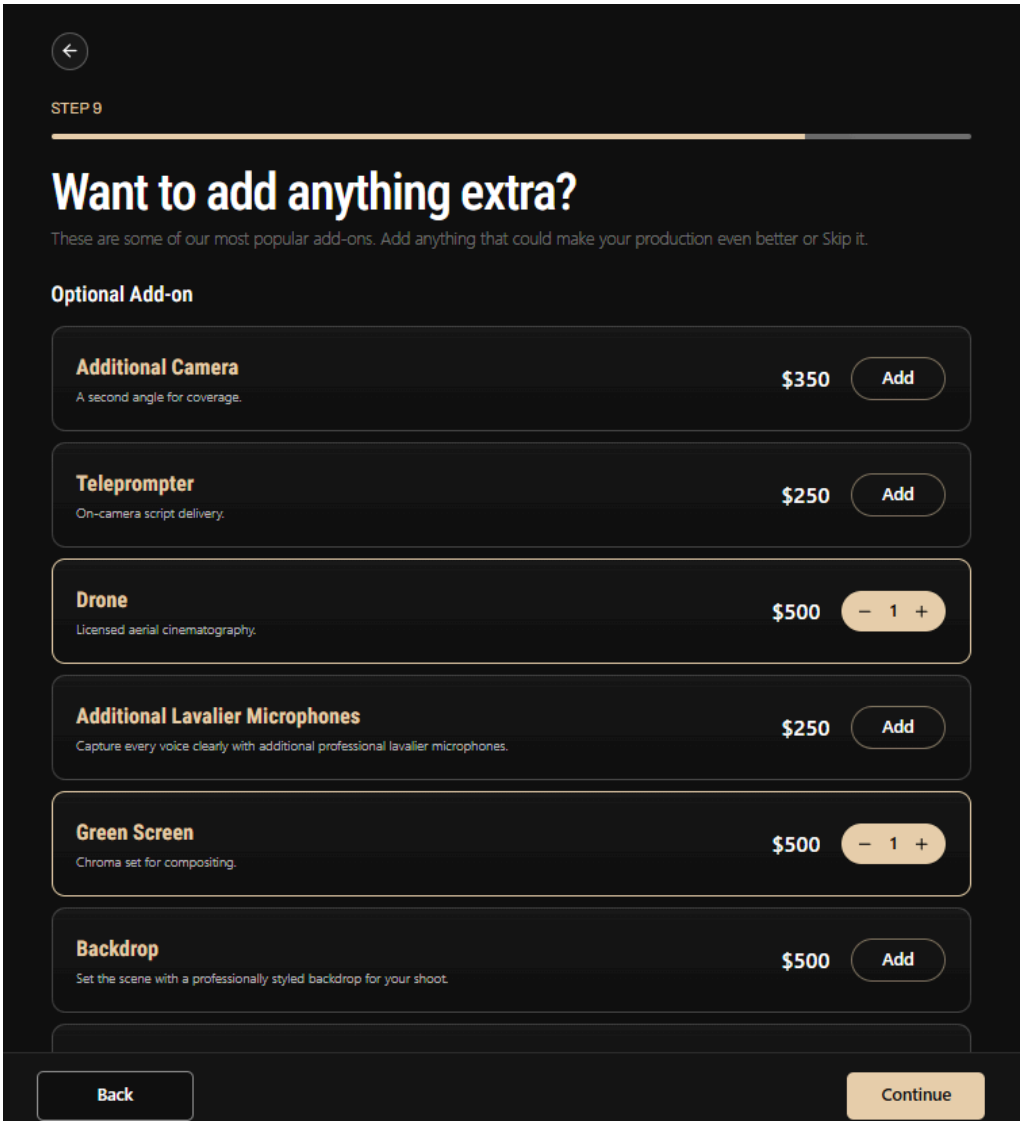

```
{
  "full_name": "Harsh Patel",
  "email": "harsh@test.com",
  "phone": "7777777777",
  "matching_method": "ai_matchmaker",
  "selected_crew_ids": [2129, 2130]
}
```


For Let Beige Choose with zero manually selected creators:


```
{
  "full_name": "Harsh Patel",
  "email": "harsh@test.com",
  "phone": "7777777777",
  "matching_method": "ai_matchmaker",
  "selected_crew_ids": []
}
```


The UI role assignment is only for showing correct Photographer/Videographer counts and validating Continue. The backend receives only final selected creator IDs, exactly like the web flow.

Screen 9 — Add-ons

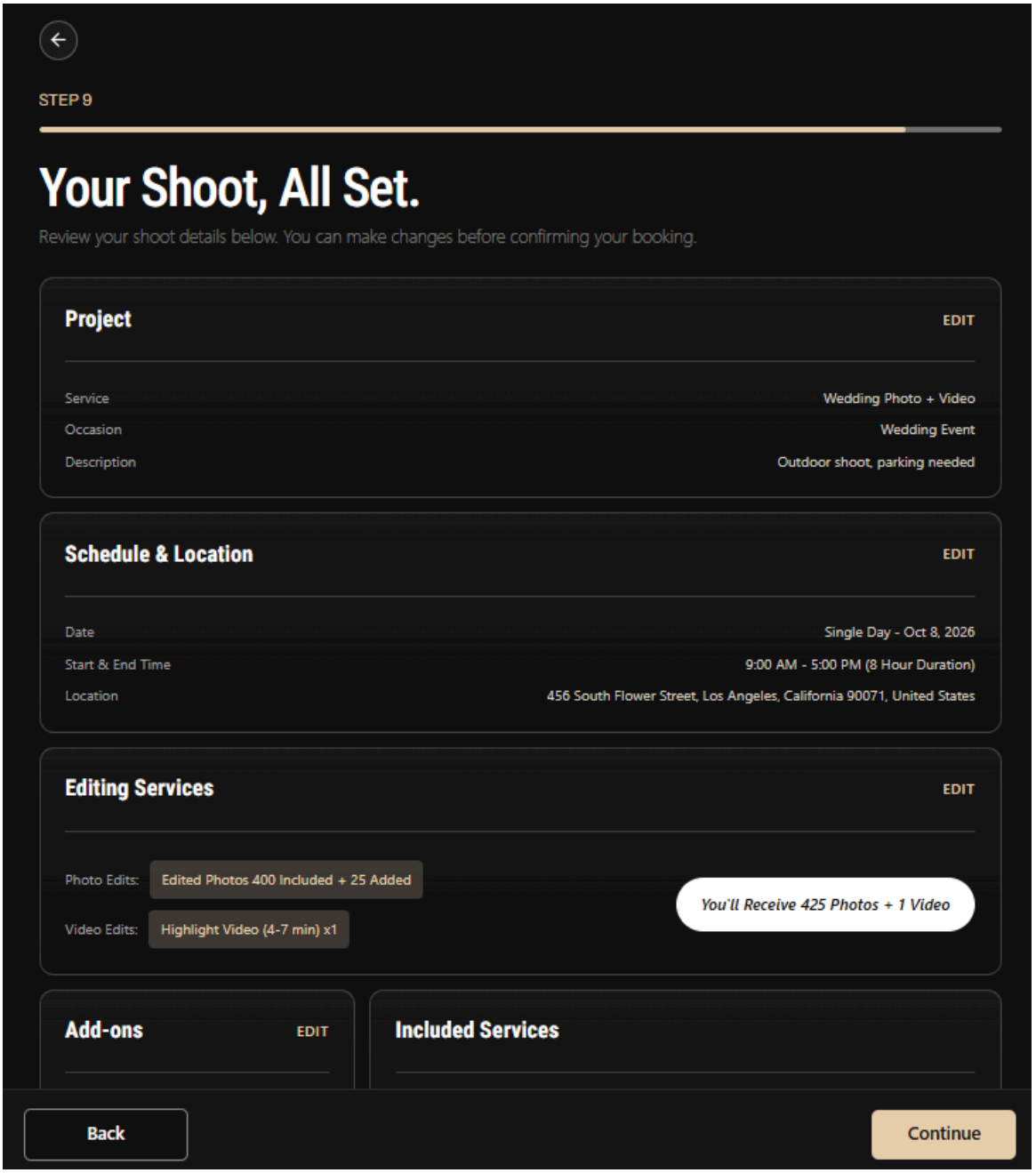


This is the same as Branch A Screen 8.

GET /bookings/addons

Keep selected add-ons in frontend state.

Screen 10 — Review and Contact Information



This screen works exactly like Branch A Screen 9.

The only difference is its step number. It is Screen 10 because Screen 8 was Creative Partner Selection and Screen 9 was Add-ons.

Load Screen 10 data

POST /bookings/:bookingId/review

```
{
  "add_on_items": [
    {
```

```
    "slug": "v4-additional-camera",
    "quantity": 1
  }
 ]
}
```

Use the same direct response mapping:

UI field	Response field
Service	`data.project.service`
Occasion	`data.project.occasion`
Description	`data.project.description`
Schedule and Location	`data.schedule.*`
Photo Edits	`data.editing_services.photo_edits`
Video Edits	`data.editing_services.video_edits`
You'll Receive	`data.editing_services.receive_text`
Add-ons	`data.add_ons`
Included Services	`data.included_services`
Contact values	`data.contact.*`

When user taps Continue

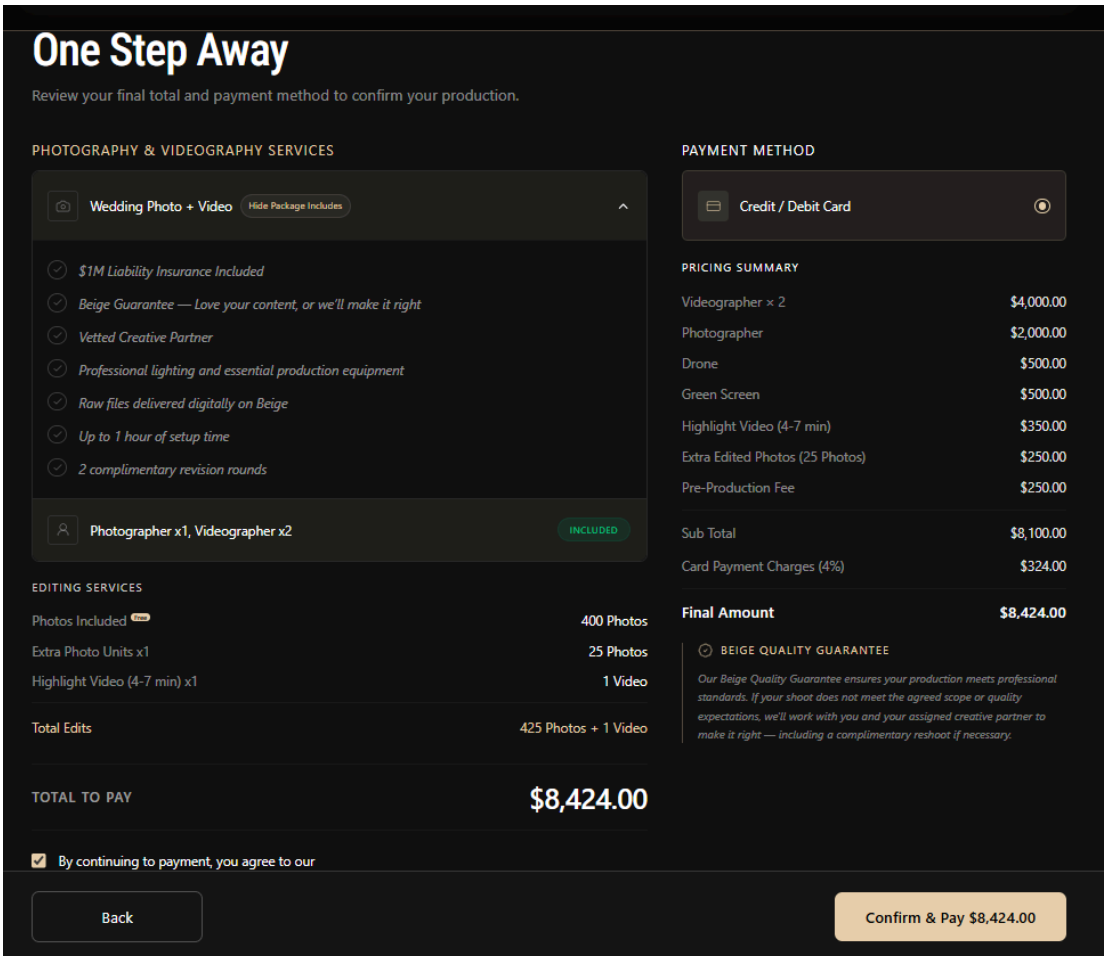
POST /bookings/:bookingId/quote

```
{
 "add_on_items": [
  {
   "slug": "v4-additional-camera",
   "quantity": 1
  }
 ]
```

}

After quote save succeeds, open Screen 11 — Confirm and Pay.

Screen 11 — Confirm and Pay



When this screen opens

Call:

GET /bookings/:bookingId/payment-review

Use the same direct response mapping as Branch A Screen 10:

data.package.service

data.package.included_services

data.package.crew

data.editing_services.photo_edits

data.editing_services.video_edits

data.editing_services.receive_text

data.pricing.line_items

data.pricing.subtotal

data.pricing.card_processing_fee

data.pricing.final_amount

data.payment_method.label

When user taps Confirm & Pay

First save contact information and the selected Creative Partner IDs:

PUT /bookings/:bookingId/submit

```
{
  "full_name": "Harsh Patel",
  "email": "harsh@test.com",
  "phone": "7777777777",
  "matching_method": "ai_matchmaker",
  "selected_crew_ids": [2129, 2130]
}
```

`selected_crew_ids` must contain IDs selected on the Creative Partner screen.

Important:

- Do not call hold APIs.
- The backend saves selected creator IDs as final selected creators.

After `/submit` succeeds, open payment:

POST /bookings/:bookingId/paymentsheet

Stripe:

{

"payment_method": "stripe"

}

Commas:

{

"payment_method": "commas"

}

Payment Screen

The user selects Stripe or Commas.

Stripe PaymentSheet

Request a mobile Stripe PaymentSheet configuration:

POST /bookings/:bookingId/paymentsheet
{
"payment_method": "stripe"
}

Use the returned Stripe PaymentSheet data in the mobile Stripe SDK.

After the Stripe SDK reports successful payment, The backend verifies payment through the Stripe webhook.

Commas Embedded Checkout

Request Commas checkout data:

POST /bookings/:bookingId/paymentsheet
{
"payment_method": "commas"

}

Use the returned embedded checkout URL/data in the app WebView or Commas payment UI.

Payment completion is verified by the backend webhook.

Booking Summary After Payment

After payment succeeds, load the booking summary:

GET /bookings/:bookingId/summary-details

Use this response for the confirmed booking screen.
